STUDENT

0007-RMO

TENTAMEN

230821 DV2606 2110 Salstentamen (DO)

Kurskod	--
Bedömningsform	--
Starttid	21.08.2023 09:00
Sluttid	21.08.2023 14:00
Bedömningsfrist	--
PDF skapad	21.05.2024 13:17
Skapad av	Zandra Johansson

1 There are, in general, two approaches to communicate between the processors in a MIMD parallel computer: *shared-memory* and *message-passing*.

- Describe the main characteristics of each of the two approaches.
- Further, present some advantage and disadvantage for each of them.
- Finally, give some example of parallel applications (with motivation) where shared-memory and message-passing would be preferable over the other, respectively.

Shared-memory systems typically communicate by using shared variables. As the name suggest, processes in these systems have a shared memory, which means that written values of one process can be read by another. These systems are typically executed with threads, since these use a shared address space.

Message-passing is a communication method typically used in distributed systems. Processors in this system cannot communicate directly via reading variables since they have separate local memories. To pass information between each other, processors have to explicitly call a communication function, and then send data in whats called a message.

Advantage of shared memory systems is its speed and simplicity. The programmer doesn't have to worry about using communication operations for every data transfer. Also since the data doesn't have to be transferred over different local memories, the communication becomes faster.

Advantage for distributed systems is scalability. Having the requirement that all data is stored in the same memory limits the amount of memory that the system can use. In a distributed system, you could connect a much larger amount of memories in a network.

Shared memory application: Smaller applications where the data can fit in local memory. Then it becomes advantageous to store the data in the same memory and utilize quicker access times and simpler communication.
Distributed memory application: Larger application which has large memory requirements. It could be a scientific simulation running on a cluster of computers. Then it is a distributed system and they will need to communicate with message passing.

Ord: 251

Besvarad.

2 Describe (define) what we mean with *speedup* and *efficiency*, in the context of a parallel computer system.

Speedup is the relative performance of a parallel system of a program in comparison to the best sequential version of that program. It is calculated by the formula T*(n)/Tp(n), where T*(n) is the sequential run time of the best sequential algorithm and Tp(n) is the parallel run time. Thus if the run times are 1 second and 0.2 second for the sequential and parallel computers, the speedup will be 1/0.2 = 5.

Efficiency relates to the cost of the parallel program. It is a measure of how much total work is done by the parallel system in comparison to the best sequential algorithm. If a parallel program has to perform a lot of redundant work that the sequential version don't do, the efficiency decreases. It is defined by the formula: T*(n)/C(n) where C(n) is the cost. The cost measures the amount of work in the following way: Tp(n)*n, where Tp(n) is the parallel processing time and n is the number of processors. So when there is a linear speedup of s=n, this means that T*(n)=Tp(n)*n and the efficiency is 1.

Ord: 180

Besvarad.

3 When we parallelize a sequential application, e.g., by using threads or message-passing, so it can can execute in parallel on *N* processors, we often find that the parallel program is not *N* times faster. Describe two important reasons for that.

One reason is communication overhead, since parallel processor may have to transfer data between them, which a sequential computer doesn't have to do, the extra wait times incurred by communication operations adds extra time to the execution.

Another is synchronization. To avoid data races caused by multiple threads accessing the same memory location, it may be necessary to use synchronization operations like mutex locks. When locking away a critical section in the code, this section becomes sequentialized, and the other processors have to idle while waiting for the owner of the mutex to finish its execution. There are other ways synchronization can cause idling, for example having a barrier operation, which means some threads may have to wait for the others to finish.

Ord: 123

Besvarad.

4 Sometimes we find that a system or application has a *superlinear speedup*. What does that mean?
Describe two situations when it may occur.

The theoretical limit on the speedup of a parallel system is having a speedup that's equal to the number of processors, this is called a linear speedup.
However, during some circumstances the speedup could be even higher, which is called a superlinear speedup. Two cases in which this could occur:

1. Due to cache effects. When partitioning the data between a number of processors, the data partitions could be small enough to fit in a single cache line. This causes the cache hit rate to increase, which means less wasted time in retrieving data from the memory, which can significantly increase performance.

2. Due to the nature of the algorithm. In some applications, having more processors working simultaneously on a problem can lead to the solution being found faster, and thus having the program terminate faster. One example of this is a search algorithm. Instead of having a single processor do a depth-first search of the entire tree, multiple threads could search multiple branches in parallel. This could lead to less work being done overall. If for example the search target is at the beginning of a branch to the right side of the tree, it would be found quickly by one thread searching that branch. But a sequential version would have to search through all the leftward branches first (if we use Depth-first search).

Ord: 225

Besvarad.

5 *Cache coherence* is an important issue to handle in a shared-memory multiprocessor.
Describe, by giving an example, why ensuring cache coherence is important. (1p)
Two main approaches exist to maintain cache coherence: *write-invalidate* and *write-update*. Briefly describe each of them, e.g., by drawing a simple picture. (2p)
Describe some advantage and disadvantage with each of the approaches. (1p)

Cache coherence protocols ensure that the last value written to a memory location is the value that is read by all processors accessing that memory location. If cache coherence is not maintained, different processors could be reading different values of the same memory location, which can lead to erroneous or unexpected behavior.

One example: Processors are called Pn and caches Cn. P1 reads 3 from memory location x1 and stores it in C1. P2 reads 3 from the same memory location and stores it in C2. P1 modifies x1 to 5, and stores the modified value in C1. P2 reads x1 again, but since its stored in C2, it reads its local value which is still 3.

Two approaches to maintaining cache coherence.
Write invalidate: Every time a memory location is modified, the cache it is stored in is marked as modified. All other caches that store a local copy of this memory location is marked as invalidated. If a processor tries to retrieve a value from a cache marked as invalidated, it has to be replaced first to get the most recent values.

Write-update: Every time a memory location that is stored in a cache is changed, all caches that store a local copy from that memory location is updated to match this so all caches have the most recent values.

The tradeoffs between these two different protocols is one between having more cache misses, or more traffic on the data buses.
Updating all caches after every write leads to more traffic on the data buses. If there is a large number of caches in the system, this could lead to performance degradation due to limited bandwidth on the data buses. The upside however is that all caches have the most recent value, then there will be less cache misses (invalidation misses) caused by having invalid caches, which means other processors wont have to waste time retrieving new cache lines.
The opposite is true for the write-invalidate approach. Less traffic used on the data buses since all caches no longer have to be written to after each individual update on a cache, only invalidation signals have to be sent. But now the other caches that have invalidated caches will have to switch them out the next time they want to access them.

Ord: 383

Besvarad.

6 When writing a parallel application, we can use mainly two approaches: *task (a.k.a. functional) parallelism* and *data parallelism*.

- Describe the main characteristics of each of the two approaches.
- Further, describe some advantage and disadvantage with each of them.
- Finally, give some examples of applications (with motivation) that are suitable for task parallelism and data parallelism, respectively.

The characteristics of data parallelism is that you divide up the computations based on the data. You have each thread be responsible for one portion of the data. For example in image processing each thread could independently process different segments of the image.

For functional parallelism you instead divide up the computations based on the different tasks that needs to be performed. These tasks could be entirely different in nature, while data decomposition typically results in the same instructions being performed on different data.

Advantages of data parallelism is that they typically result in simpler synchronization, since the same instructions are usually performed on different data parts independently. Different tasks on the other hand can have dependencies between them, and be of different length which can make it harder to synchronize them.

Disadvantage of data parallelism is that it depends on being able to find some partition of data that can work independently which may not be possible for all applications, while in task parallelism you have more flexibility on choosing which tasks should be made in parallel.

Data parallelism example: Vector addition. You can divide up the array vector into a set of sections, where the addition can be calculated by each thread on that section independently from the others.

Task parallelism example: A server working on multiple and potentially different tasks from a set of clients simultaneously.

Ord: 229

Besvarad.

7 Explain how a barrier synchronization works in parallel systems, including why it sometimes is needed, and some possible drawbacks with barriers.

The idea of a barrier synchronizaiton is to set up a barrier in a specific location of the code where all executing processors have to reach before any processor can proceed past that point. If processor P1 reaches the barrier, but P2 has not, then P1 will have to wait for P2.

Barriers can be needed when there are dependencies in the code that needs to be fulfilled before the next phase of the program can proceed. One example is when threads cooperate with loading in data concurrently. Before the execution of the program can proceed, all the required data needs to be loaded in first. Otherwise some thread could execute calculations on the wrong data since it didn't arrive before this thread reached its calculation stage.
This could be fixed by setting up a program like this:
load_data()
barrier_synchronize()
compute_data()

A drawback to barriers is that it is a bottleneck that can cause performance issues. If most threads finish execution early but have wait for some lingering thread, then this can cause a lot of wasted time by having processors idling while waiting for the other.

Ord: 187

Besvarad.

8

Consider the sequential C code below for performing a matrix transpose operation.

```
1  #define SIZE 2048
2  static int a[SIZE][SIZE];
3
4  static void init_a()
5  {
6      ... // init matrix a
7  }
8
9  static void transpose_seq()
10 {
11     int m, n, tmp;
12     for (m = 0; m < SIZE-1; m++)
13         for (n = m+1; n < SIZE; n++) {
14             // swap a[m][n] and a[n][m]
15             tmp = a[m][n];
16             a[m][n] = a[n][m];
17             a[n][m] = tmp;
18         }
19 }
20
21 int main(int argc, char **argv)
22 {
23     init_a();
24     transpose_seq();
25 }
```

Write (in C-like code) a parallel version of the transpose algorithm, i.e., parallelize the function **transpose_seq()**. Clearly state which assumptions you do about the type of machine and programming model. Assume that the number of processors is significantly fewer than SIZE.(2p)

Analyze your solution from a performance point of view, e.g., approximate speed-up, load-balancing, etc. (2p)

Each iteration of the outer loop can be calculated independently from the others. Therefore the outer loop is replaced instead by a loop that creates threads and then makes each thread calculate one iteration of the loop.

Assumptions: Machine has 16 processor cores and runs on a linux system which has access to pthreads library.

The code works by dividing up the outer loop which iterates through m from 0 to SIZE, equally between the 16 processors. This means each processor gets to work on a chunk of the loop which has the size of SIZE/16. Thread one takes iterations m 0->127, thread 2 takes m 128-255, etc...

The speed up will approximately match the number of processors which is 16, since there are 16 processor working on equal number of iterations simultaneously. But the speedup will be a bit less, maybe 10-14. Partly because of the overhead costs from thread management, like creation and joining of threads. But also because of load imbalance. The second loop gets smaller for each iteration of n, since it starts at n=m+1. This means that threads that take on smaller m-values will have more work to do. So the first thread taking m iterations 0-127 will do a lot more work than the final thread which takes the last 128 iterations. One way to reduce this problem would be to instead divide up the iterations in a strided pattern. I.e thread 0 takes m=0, thread 1 takes m=1, thread k takes m= k mod(m), etc... But i didnt have time to implement this.

#define SIZE 2048

static int a[SIZE][SIZE;

```
int chunk_size = 2048/16

static void init_a()
{
	...//init matrix a
}

void transpose_parallel( *arg)
{

	int i = *arg;
	int m,n, tmp

	start_index = i*chunk_size;
	end_index = start_index + chunk_size;

	for (m = start_index ; m < end_index ; m++)
		for (n = m+1; n < end_index ,  n++ ) {
		 //swap a[m][n] and a[n][m]
		tmp = a[m][n]
		 a[m][n] = a[n][m]
		  a[n][m] = tmp;
		 }
}

int main( int argc, char **argv)
{
	 init_a();

	pthread_t threads[16];

	for (i = 0 ; i < 16 ; i++) {
		int* thread_arg = i
		pthread_create( threads[i], transpose_parallel, thread_arg);
	}

	for (i = 0; i < 16; i++) {
		pthread_join( threads[i]);
	}
}
```

Ord: 378

Besvarad.

9 Regulate padding

The Coordinate (COO) format makes it possible to provide regularization (by regulating padding) when working with sparse matrices. Explain with a small example how the COO format can provide regularization when working with sparse matrices.

The ELL format utilizes padding. The row with the maximum number of nonzero values determines the size of each row in the matrix. All the rows that have less nonzero values than the maximum, gets padded to match the size of the maximum. This can however cause control divergence and space inefficiencies. If one or a few number of outlier rows have a much larger number of nonzero values than the rest, then the other rows will have many padded elements. This format can be regularized by combining it with the COO format, it then becomes a hybrid ELL-COO format. These outlying rows could instead be stored in COO, while the rest still uses the ELL format. This could greatly reduce the number of padded elements, while retaining the benefits of ELL (like memory coalescing) on most of the data.

Ord: 140

Besvarad.

10 Atomic operations

What is an atomic operation in CUDA, and in what cases do we need atomic operations? Please give a small example.

An atomic operation is one that gets completed without having other threads interfering during the operation. it could for example be an atomic read and write operation, where one process reads a value from a memory location, updates this value and then writes it back to the location, without having any other process access that memory location during this operation, forcing them to wait for the atomic operation to finish.

Atomic values can be needed when multiple threads are updating the same memory location. One example is during histogram calculation. You can have each thread count some values for different portions of the data. The counters are the same for the entire data, so the threads will have to update shared counter variables. If the counters are not stored privately by each thread, the counters will be needed to be updated with atomic operations. A process has to read the current count value, increment it by one and then write it back again. If not atomic, erroneous execution can happen due to data races. For example, two different threads could read the counter value at the same time. Both threads increment the same value by one, and then stores this value to the same memory location. The effect of this is that one increment is lost, since what was supposed to be two increments became one.

Ord: 226

Besvarad.

11 Cache versus shared memory

When programming GPUs one can sometimes use either the cache or the shared memory to reduce the global memory bottleneck problem. Give one example of a situation where placing a data structure in either the cache or the shared memory would improve performance compared to keeping the data structure in global memory. Also, discuss the advantages and disadvantages of using the cache instead of shared memory.

Data structures that are shared by multiple threads can be stored in shared memory or cache to your benefit. If a data structure is stored in a shared memory that is used by multiple threads, this means that the number of memory loads from global memory can be reduced. Since otherwise each thread would have to load in the element from global memory separately. This causes redundant loads since the threads would be loading in the same memory values, which instead could be accessed from a shared data structure. Example applications are matrix multiplication and convolution computation where arrays can be loaded in to shared memory and be used by multiple threads. This improve performance because the memory latency from accessing global memory is very slow in comparison to local operations.

The disadvantage of cache is that the storage there is not persistent. If a data structure is not used for a while, there is a risk that it gets removed from the cache when a cache line needs to get replaced. The threads will then have to load in the data structure again if it wants to use it. In a shared memory you can count on the stored data structure to stay as long as needed. The advantage of caches is that the hardware through some policy (like least recently used) automatically stores data that is used often, without the programmer having to infer what data structures will be used most and explicitly load them in.

Ord: 248

Besvarad.

12 Global memory bandwidth

The memory bandwidth to global memory is a performance bottleneck in most computer systems. Two techniques for reducing the memory bandwidth bottleneck in GPUs are using cache memory and shared memory. Explain at least two other techniques that a GPU uses to reduce the global memory bandwidth problem.

One technique is accumulation. This means that updates to a memory location are accumulated into a larger update that gets sent as one update, instead of sending several smaller updates. This reduces the number of memory operations performed, since each update requires an access to the memory location. For example, in a histogram algorithm, the increments to the counters could be accumulated so a single update is sent to the counter which is the sum of all increments.

Another technique is thread coarsening. You can make each individual threads perform a larger share of the computations, resulting in fewer threads and memory operations. For example, instead of having 10 threads perform 10 load operations from the global memory, you could have 5 threads perform 5 load operations that are twice as large.

Ord: 132

Besvarad.

13 Scatter to gather conversion

Scatter to gather conversion is a technique that can be used for improving the performance of some CUDA program. Explain how scatter to gather conversion works and why this technique can improve performance.

Scatter to gather conversion works by changing the execution of the algorithm to be input oriented to output oriented. When using a scatter approach, you iterate through all the input elements, perform some calculation and scatter the results to all output elements that are affected by this input. In the gather approach, it works in the opposite way. You iterate through the output elements, and gather the values from all the input elements that affect this output.

The input elements could for example be atoms with a certain charge, and the outputs could be grid points that store some combined energy value caused by all atoms in the system. You can either iterate through all atoms (inputs) or iterate through all grid points (outputs).

Here is why gather can improve performance. If all threads work on separate inputs in a scatter approach, they will have to store their results on an output element that all other threads also have store their results in. This means that the output elements become contested areas, and atomized operations to read and write to these outputs will be have to be performed. But atomized operations should be avoided whenever possible, because this causes a loss of performance since a sequentialization of operations on these memory locations have to be made. If you instead use a gather approach, each output element could instead be stored privately by each thread, and all input elements could be gathered independently from all input elements without any problems incurred from data race, since the inputs are constant.

Ord: 258

Besvarad.

14 Cuda Streams

What are Cuda Streams? How can they be used to improve performance? Give a short example.

A Stream in CUDA is a sequence of operations that can be specified by the host and then sent to the GPU for execution. These streams can be executed asynchronously by the GPU. A way to increase performance with these streams is to use them to combine data transfer and computation at the same time in the GPU. When using a single stream of execution, the GPU has to either perform some computation operation or data transfer operation. While the GPU is computing something, the data buses would be idle, and during a data transfer the execution resources would be idle. The performance could be increased by combining both data transfer and computation, keeping more resources of the GPU busy. An example application is when performing some grid computation, neighboring nodes could need access to parts of the local data of a node, called halo element. To maximize performance, each node would want to send out the required halo elements to it neighbors, while simultaneously performing calculation on its internal data. This can be achieved by deploying two CUDA streams, one for data transfer between the nodes and one for internal node computation.

Ord: 193

Besvarad.

15 Tiling

Tiling is a technique that can be used for improving the performance of some CUDA programs, e.g., programs that perform parallel matrix multiplication or convolution. Explain how tiling works, why it can increase performance and why and how we may need to use __synchthreads() when using tiling.

The idea of tiling is to store memory locations that are used by multiple threads in shared memory. The values stored in the tile could be reused by multiple threads, instead of having threads redundantly load in the same value separately. This causes fewer load operations to access the global memory. Since memory latency can be a bottleneck in performance, reducing the number of memory accesses can greatly increase performance. In addition, after the elements have been loaded in, access time to shared memory is much faster compared to accessing global memory

In the case of matrix multiplication, a tile could for example be a 4x4 subsection of the matrix that gets loaded in to the shared memory. All output elements on the same row access the same row of data, thus the same value gets reused four times. This leads to 4x less memory operations, in comparison to having each output element thread load in the values required separately.

When using tiling, the task of loading in the data can be made more efficient by distributing this task between the threads, i.e each thread loads in some portion of the data. This is where the __synchthreads() operation is necessary. This is a barrier operation that ensures that all threads will have to finish before proceeding with the execution. In this case, we don't want any threads to start computation before the data has been loaded in. that's why a __synchthreads() operation can be put after the memory load operation, to ensure that all the memory has been loaded in before the computation starts.

Ord: 264

Besvarad.

16

What is thread divergence and why would we like to avoid thread divergence? Give an example of how thread divergence can be avoided or reduced.

Thread divergence is having different threads executing different branches of the code. This can be problematic in GPUs since they are optimized to run the same instructions in parallel. All the threads in a warp, which consists of 32 threads, have to run the same instruction. If these threads have diverging execution paths, then their execution will have to be divided up between the scheduling of the warps. This causes a loss of parallelism and performance.

One example of how thread divergence is avoided is the use of sparse matrix formats that are designed to reduce thread divergence. One of these formats is JDR. It works by sorting the rows of the matrix in terms of how the number of nonzero elements, e.g having rows with the largest number of nonzero elements at the top (and then it applies some transposition of the matrix). This causes adjacent rows in the matrix to have similar number of nonzero elements. If you give adjacent rows to adjacent threads, this in turn means that all threads in a warp will process rows with a similar number of nonzero elements. This means that all threads in the warp will execute similar number of instructions, and thus will reduce the thread divergence.

Ord: 207

Besvarad.